

```
CREATE DATABASE IF NOT EXISTS employeeedb1;
USE employeeedb1;
```

```
CREATE TABLE IF NOT EXISTS employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary DECIMAL(10,2)
);
```

```
INSERT INTO employees VALUES
(101, 'Rahul', 1, 50000),
(102, 'Priya', 2, 65000),
(103, 'Anil', 1, 55000),
(104, 'Sneha', 3, 70000),
(105, 'Kiran', 2, 48000);
```

```
-- Verify Data
SELECT * FROM employees;
```

```
CREATE TABLE IF NOT EXISTS departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

```
INSERT IGNORE INTO departments VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'EEE');
```

-- Procedure 1: Display All Employees

```
DROP PROCEDURE IF EXISTS GetEmployees;
DELIMITER $$
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT * FROM employees;
END $$
DELIMITER ;
CALL GetEmployees();
```

-- Procedure 2: IN Parameter

```
DROP PROCEDURE IF EXISTS GetEmployeeById;
DELIMITER $$
CREATE PROCEDURE GetEmployeeById(IN eid INT)
BEGIN
    SELECT * FROM employees
    WHERE emp_id = eid;
END $$
DELIMITER ;
CALL GetEmployeeById(102);
```

-- Procedure 3: Update Salary

```
DROP PROCEDURE IF EXISTS IncreaseSalary;
DELIMITER $$
CREATE PROCEDURE IncreaseSalary(
    IN eid INT,
    IN amount DECIMAL(10,2)
```

```

)
BEGIN
    UPDATE employees
    SET salary = salary + amount
    WHERE emp_id = eid;
END $$
DELIMITER ;
CALL IncreaseSalary(101,5000);
SELECT * FROM employees WHERE emp_id=101;

```

-- Procedure 4: Aggregate Function

```

DROP PROCEDURE IF EXISTS AvgSalary;
DELIMITER $$
CREATE PROCEDURE AvgSalary()
BEGIN
    SELECT AVG(salary) AS AverageSalary
    FROM employees;
END $$
DELIMITER ;
CALL AvgSalary();

```

-- Procedure 5: JOIN

```

DROP PROCEDURE IF EXISTS EmployeeDepartment;
DELIMITER $$
CREATE PROCEDURE EmployeeDepartment()
BEGIN
    SELECT e.emp_name, d.dept_name, e.salary
    FROM employees e
    JOIN departments d ON e.dept_id=d.dept_id;
END $$
DELIMITER ;
CALL EmployeeDepartment();

```

-- Procedure 6: GROUP BY and AVG

```

DROP PROCEDURE IF EXISTS DepartmentAverageSalary;
DELIMITER $$
CREATE PROCEDURE DepartmentAverageSalary()
BEGIN
    SELECT d.dept_name,
        COUNT(e.emp_id) AS EmployeeCount,
        AVG(e.salary) AS AverageSalary
    FROM employees e
    JOIN departments d ON e.dept_id=d.dept_id
    GROUP BY d.dept_id, d.dept_name;
END $$
DELIMITER ;
CALL DepartmentAverageSalary();

```

-- Procedure 7: IF...ELSE

```

DROP PROCEDURE IF EXISTS SalaryStatus;
DELIMITER $$
CREATE PROCEDURE SalaryStatus(IN eid INT)
BEGIN
    DECLARE sal DECIMAL(10,2);
    SELECT salary INTO sal
    FROM employees

```

```

WHERE emp_id=eid;
IF sal >= 60000 THEN
    SELECT 'High Salary' AS Status;
ELSE
    SELECT 'Low Salary' AS Status;
END IF;
END $$
DELIMITER ;
CALL SalaryStatus(102);

```

-- Procedure 8: OUT Parameter

```

DROP PROCEDURE IF EXISTS EmployeeCount;
DELIMITER $$
CREATE PROCEDURE EmployeeCount(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total
    FROM employees;
END $$
DELIMITER ;
CALL EmployeeCount(@total);
SELECT @total AS TotalEmployees;

```

-- Procedure 9: INOUT Parameter

```

DROP PROCEDURE IF EXISTS AddBonus;
DELIMITER $$
CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
BEGIN
    SET bonus = bonus + 5000;
END $$
DELIMITER ;
SET @bonus=10000;
CALL AddBonus(@bonus);
SELECT @bonus AS UpdatedBonus;

```

-- Procedure 10: Transaction

```

DROP PROCEDURE IF EXISTS TransferSalaryAmount;
DELIMITER $$
CREATE PROCEDURE TransferSalaryAmount(
    IN fromEmp INT,
    IN toEmp INT,
    IN amount DECIMAL(10,2)
)
BEGIN
    START TRANSACTION;
    UPDATE employees
    SET salary=salary-amount
    WHERE emp_id=fromEmp;
    UPDATE employees
    SET salary=salary+amount
    WHERE emp_id=toEmp;
    COMMIT;
END $$
DELIMITER ;
CALL TransferSalaryAmount(101,102,2000);
SELECT * FROM employees WHERE emp_id IN (101,102);

```

-- Task 1: Create a procedure to display employees whose salary is greater than a given amount.

```
DROP PROCEDURE IF EXISTS GetEmpByMinSalary;
DELIMITER $$
CREATE PROCEDURE GetEmpByMinSalary(IN min_amount DECIMAL(10,2))
BEGIN
    SELECT * FROM employees WHERE salary > min_amount;
END $$
DELIMITER ;
CALL GetEmpByMinSalary(60000);
```

-- Task 2: Create a procedure to update an employee's department using IN parameters.

```
DROP PROCEDURE IF EXISTS UpdateEmpDepartment;
DELIMITER $$
CREATE PROCEDURE UpdateEmpDepartment(IN eid INT, IN new_dept INT)
BEGIN
    UPDATE employees SET dept_id = new_dept WHERE emp_id = eid;
END $$
DELIMITER ;
CALL UpdateEmpDepartment(105, 1);
```

-- Task 3: Create a procedure to return the maximum salary using an OUT parameter.

```
DROP PROCEDURE IF EXISTS GetMaxSalary;
DELIMITER $$
CREATE PROCEDURE GetMaxSalary(OUT max_sal DECIMAL(10,2))
BEGIN
    SELECT MAX(salary) INTO max_sal FROM employees;
END $$
DELIMITER ;
CALL GetMaxSalary(@max);
SELECT @max AS MaximumSalary;
```

-- Task 4: Create a procedure to return the employee count of a specified department.

```
DROP PROCEDURE IF EXISTS GetDeptEmpCount;
DELIMITER $$
CREATE PROCEDURE GetDeptEmpCount(IN d_id INT, OUT emp_count INT)
BEGIN
    SELECT COUNT(*) INTO emp_count FROM employees WHERE dept_id = d_id;
END $$
DELIMITER ;
CALL GetDeptEmpCount(1, @count);
SELECT @count AS Dept1_EmployeeCount;
```

-- Task 5: Create a procedure using JOIN to display employee and department details.

```
DROP PROCEDURE IF EXISTS GetEmpAndDeptDetails;
DELIMITER $$
CREATE PROCEDURE GetEmpAndDeptDetails()
BEGIN
    SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id;
END $$
DELIMITER ;
CALL GetEmpAndDeptDetails();
```

-- Task 6: Create a procedure to display departments whose average salary is greater than 55000.

```
DROP PROCEDURE IF EXISTS GetHighAvgSalDepts;
DELIMITER $$
CREATE PROCEDURE GetHighAvgSalDepts()
BEGIN
    SELECT d.dept_name, AVG(e.salary) AS AvgSal
    FROM employees e
    JOIN departments d ON e.dept_id = d.dept_id
    GROUP BY d.dept_id, d.dept_name
    HAVING AvgSal > 55000;
END $$
DELIMITER ;
CALL GetHighAvgSalDepts();
```

-- Task 7: Create a procedure using IF...ELSE to classify salary as High, Medium or Low.

```
DROP PROCEDURE IF EXISTS ClassifySalaryLevel;
DELIMITER $$
CREATE PROCEDURE ClassifySalaryLevel(IN eid INT)
BEGIN
    DECLARE sal DECIMAL(10,2);
    SELECT salary INTO sal FROM employees WHERE emp_id = eid;

    IF sal >= 65000 THEN
        SELECT 'High' AS SalaryLevel;
    ELSEIF sal >= 55000 THEN
        SELECT 'Medium' AS SalaryLevel;
    ELSE
        SELECT 'Low' AS SalaryLevel;
    END IF;
END $$
DELIMITER ;
CALL ClassifySalaryLevel(103);
```

-- Task 8: Create a procedure that accepts a department ID and returns its average salary.

```
DROP PROCEDURE IF EXISTS GetAvgSalByDept;
DELIMITER $$
CREATE PROCEDURE GetAvgSalByDept(IN d_id INT, OUT avg_sal DECIMAL(10,2))
BEGIN
    SELECT AVG(salary) INTO avg_sal FROM employees WHERE dept_id = d_id;
END $$
DELIMITER ;
CALL GetAvgSalByDept(2, @avg);
SELECT @avg AS ECE_Average_Salary;
```

-- Task 9: Create a procedure using INOUT to apply a bonus to an input salary.

```
DROP PROCEDURE IF EXISTS ApplyCustomBonus;
DELIMITER $$
CREATE PROCEDURE ApplyCustomBonus(INOUT current_sal DECIMAL(10,2), IN bonus
DECIMAL(10,2))
BEGIN
    SET current_sal = current_sal + bonus;
END $$
DELIMITER ;
SET @my_sal = 50000;
```

```
CALL ApplyCustomBonus(@my_sal, 2500);
SELECT @my_sal AS SalaryAfterBonus;
```

-- Task 10: Create a procedure that performs two related updates inside a transaction.

```
DROP PROCEDURE IF EXISTS TransferEmployeeDeptAndRaise;
DELIMITER $$
CREATE PROCEDURE TransferEmployeeDeptAndRaise(IN eid INT, IN new_dept INT, IN
raise_amount DECIMAL(10,2))
BEGIN
    START TRANSACTION;
    UPDATE employees SET dept_id = new_dept WHERE emp_id = eid;
    UPDATE employees SET salary = salary + raise_amount WHERE emp_id = eid;
    COMMIT;
END $$
DELIMITER ;
CALL TransferEmployeeDeptAndRaise(101, 2, 3000);
SELECT * FROM employees WHERE emp_id = 101;
```

-- Procedure 1: Display All Employees

	emp_id	emp_name	dept_id	salary
	101	Rahul	2	59000.00
	102	Priya	2	69000.00
	103	Anil	1	55000.00
	104	Sneha	3	70000.00
	105	Kiran	1	48000.00

-- Procedure 2: IN Parameter

	emp_id	emp_name	dept_id	salary
	102	Priya	2	69000.00

-- Procedure 3: Update Salary

emp_id	emp_name	dept_id	salary
101	Rahul	2	59000.00
NULL	NULL	NULL	NULL

-- Procedure 4: Aggregate Function

AverageSalary
60200.000000

-- Procedure 5: JOIN

emp_name	dept_name	salary
Rahul	ECE	59000.00
Priya	ECE	69000.00
Anil	CSE	55000.00
Sneha	EEE	70000.00
Kiran	CSE	48000.00

-- Procedure 6: GROUP BY and AVG

dept_name	EmployeeCount	AverageSalary
ECE	2	64000.000000
CSE	2	51500.000000
EEE	1	70000.000000

-- Procedure 7: IF...ELSE

Status
High Salary

-- Procedure 8: OUT Parameter

TotalEmployees
5

-- Procedure 9: INOUT Parameter

UpdatedBonus
15000.00

-- Procedure 10: Transaction

emp_id	emp_name	dept_id	salary
101	Rahul	2	59000.00
102	Priya	2	69000.00
NULL	NULL	NULL	NULL

-- Task 1: Create a procedure to display employees whose salary is greater than a given amount.

emp_id	emp_name	dept_id	salary
102	Priya	2	69000.00
104	Sneha	3	70000.00

-- Task 2: Create a procedure to update an employee's department using IN parameters.

emp_id	emp_name	dept_id	salary
102	Priya	2	69000.00
104	Sneha	3	70000.00

-- Task 3: Create a procedure to return the maximum salary using an OUT parameter.

MaximumSalary
70000.00

-- Task 4: Create a procedure to return the employee count of a specified department.

Dept1_EmployeeCount
3

-- Task 5: Create a procedure using JOIN to display employee and department details.

emp_id	emp_name	salary	dept_name
101	Rahul	59000.00	ECE
102	Priya	69000.00	ECE
103	Anil	55000.00	CSE
104	Sneha	70000.00	EEE
105	Kiran	48000.00	CSE

-- Task 6: Create a procedure to display departments whose average salary is greater than 55000.

dept_name	AvgSal
ECE	64000.000000
EEE	70000.000000

-- Task 7: Create a procedure using IF...ELSE to classify salary as High, Medium or Low.

SalaryLevel
Medium

-- Task 8: Create a procedure that accepts a department ID and returns its average salary.

ECE_Average_Salary
69000.00

-- Task 9: Create a procedure using INOUT to apply a bonus to an input salary.

SalaryAfterBonus
52500.00

-- Task 10: Create a procedure that performs two related updates inside a transaction.

emp_id	emp_name	dept_id	salary
101	Rahul	2	59000.00
NULL	NULL	NULL	NULL